Curriculum

```c
/**
 * @file main.c
 * @brief Permutation Generator in C, architected to safety-critical standards.
 *
 * @author Dominic Alexander Cooper (Original Algorithm)
 * @author Gemini (Architectural Refactoring)
 *
 * @details
 * This program generates all possible character combinations for a given length,
 * based on a predefined character set. It is a complete rewrite of the original
 * concept to align with principles of safety-critical software design.
 */
#include <stdio.h>
#include <stdint.h> // For fixed-width integer types like uint64_t
#include <stdbool.h> // For bool type
#include <limits.h> // For UINT64_MAX
//============================================================================
// 1. CONFIGURATION AND DATA DEFINITIONS
//============================================================================
#define MAX_PERMUTATION_LENGTH 10
static const char ALPHABET[] = {
    'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
    'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
    'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',
    'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',
    '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', ' ', '\n'
};
static const uint64_t ALPHABET_SIZE = sizeof(ALPHABET) / sizeof(ALPHABET[0]);
//============================================================================
// 2. ABSTRACT INTERFACE FOR OUTPUT (OutputSink)
//============================================================================
typedef struct {
    void* context;
    bool (*write)(void* context, const char* buffer, uint64_t size);
    bool (*write_char)(void* context, char c);
} OutputSink;
//============================================================================
// 3. CORE LOGIC (Generator)
//============================================================================
static bool safe_uint64_power(uint64_t base, uint64_t exp, uint64_t* result) {
    *result = 1;
    for (uint64_t i = 0; i < exp; ++i) {
        if (*result > UINT64_MAX / base) {
            return false;
        }
        *result *= base;
    }
    return true;
```

```
}
static bool generate_permutations(uint64_t length, OutputSink* sink) {
    uint64_t num_combinations;
    if (!safe_uint64_power(ALPHABET_SIZE, length, &num_combinations)) {
        return false;
    }
    char current_perm[MAX_PERMUTATION_LENGTH];
    for (uint64_t i = 0; i < num_combinations; ++i) {
        uint64_t temp_row = i;
        for (int64_t j = length - 1; j >= 0; --j) {
            uint64_t char_index = temp_row % ALPHABET_SIZE;
            current_perm[j] = ALPHABET[char_index];
            temp_row /= ALPHABET_SIZE;
        }
        if (!sink->write(sink->context, current_perm, length)) {
            return false;
        }
        if (!sink->write_char(sink->context, '\n')) {
            return false;
        }
    }
    return true;
}
//============================================================================
// 4. CONCRETE IMPLEMENTATION OF OUTPUTSINK (FileSink)
//============================================================================
static bool file_sink_write(void* context, const char* buffer, uint64_t size) {
    FILE* p = (FILE*)context;
    return fwrite(buffer, sizeof(char), size, p) == size;
}
static bool file_sink_write_char(void* context, char c) {
    FILE* p = (FILE*)context;
    return fputc(c, p) != EOF;
}
static bool file_sink_init(OutputSink* sink, const char* filename) {
    FILE* p = fopen(filename, "w");
    if (p == NULL) {
        return false;
    }
    *sink = (OutputSink){
        .context = p,
        .write = file_sink_write,
        .write_char = file_sink_write_char
    };
    return true;
}
static void file_sink_close(OutputSink* sink) {
    if (sink && sink->context) {
        fclose((FILE*)sink->context);
        sink->context = NULL;
    }
}
//============================================================================
// 5. SYSTEM ASSEMBLER (main)
```

```c
//============================================================================
int main(void) {
    const uint64_t permutation_length = 4;
    if (permutation_length == 0 || permutation_length > MAX_PERMUTATION_LENGTH) {
        return 1;
    }
    OutputSink file_sink;
    if (!file_sink_init(&file_sink, "system_safe.txt")) {
        perror("Error opening file");
        return 1;
    }
    bool success = generate_permutations(permutation_length, &file_sink);
    file_sink_close(&file_sink);
    if (!success) {
        return 1;
    }
    return 0;
}
```